

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	1
ВВЕДЕНИЕ	2
1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	4
1.1 Анализ предметной области	4
1.2 Анализ аналогов разрабатываемого решения	5
1.3 Описание требований к разрабатываемому решению	7
2 ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ	9
2.1 Выбор инструментов и/или средств для реализации проекта	9
2.2 Разработка проекта решения	10
3 ПРАКТИЧЕСКАЯ ЧАСТЬ	14
3.1 Разработка составных элементов	14
3.2 Установка и настройка	15
3.3 Тестирование	15
3.4 Ввод в эксплуатацию	17
3.5 Разработка сопроводительной документации	18
ЗАКЛЮЧЕНИЕ	20
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	21
ПРИЛОЖЕНИЯ	22
Приложение А - Исходный код серверной части	22
Приложение Б - SQL-сценарий базы данных	35
Приложение В - Исходный код клиентской части	39

ВВЕДЕНИЕ

Современные предприятия работают с большим количеством материальных ценностей, поставщиков, единиц измерения и первичных учетных документов. При ручном ведении складского учета возрастает риск ошибок в реквизитах поставщиков, кодах материалов, количестве, цене и итоговой стоимости поступления. Ошибки в таких данных напрямую влияют на управленческие решения, планирование закупок, финансовую отчетность и контроль остатков.

Актуальность темы определяется необходимостью автоматизации типовых операций складского учета и получения быстрых отчетов по поставщикам и материалам. Веб-приложение позволяет вынести пользовательский интерфейс в браузер, централизовать серверную обработку запросов и хранить данные в реляционной базе, где целостность обеспечивается ограничениями, внешними ключами, процедурами и триггерами.

Цель работы: разработка программного продукта назначения программы «Складской учет материалов».

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области складского учета материалов;
- проанализировать аналоги разрабатываемого решения;
- описать требования к разрабатываемому решению;
- выбрать инструменты и средства для реализации проекта;
- разработать проект решения, включая структуру системы, данные и основные сценарии;
- разработать программный продукт;
- провести функциональное тестирование приложения;
- составить отчет по работе и подготовить материалы к защите.

Объектом исследования является автоматизация процесса складского учета материалов на предприятии. Предметом исследования является веб-

приложение складского учета материалов, реализующее работу со справочниками, приходными операциями, поставщиками, отчетами и пользовательским доступом.

В качестве основных методов исследования применены анализ, синтез, сравнение и моделирование. Практическая реализация поставленной задачи соответствует основным подходам к разработке программного обеспечения: разделению ответственности между клиентом, сервером и базой данных, типизации моделей, проверке входных данных и функциональному тестированию.

Информационной базой исследования являются открытые источники, официальная документация применяемых технологий, материалы курса «Создание программного обеспечения», доступные через систему дистанционного обучения РТУ МИРЭА, а также исходный код разработанного приложения.

В данном отчете представлен процесс разработки программного продукта, в том числе теоретический обзор предметной области, анализ аналогов, технологическое проектирование системы, описание программной реализации, результаты тестирования, порядок установки и сопроводительная документация.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Анализ предметной области

Складской учет материалов является частью общего учета материально-производственных запасов организации. В рамках рассматриваемой предметной области фиксируются сведения о материалах, единицах измерения, поставщиках, сопроводительных документах и приходных операциях. Каждая единица хранения должна быть связана с конкретным материалом, поставщиком, датой поступления, документом-основанием, количеством и ценой.

Ключевая особенность предметной области состоит в большом количестве взаимосвязанных справочников. Материал имеет код, код класса, код группы, наименование и счет учета. Поставщик имеет код, наименование, ИНН, юридический адрес, банковский адрес и расчетный счет. Единицы измерения зависят от материала, а приходная операция связывает все эти сущности в единый учетный документ.

При отсутствии автоматизированной системы пользователь вынужден хранить сведения в таблицах или бумажных документах. Такой подход допускает дублирование поставщиков, ошибочные коды материалов, неверное сопоставление единиц измерения и арифметические ошибки при расчете итоговой стоимости. Кроме того, ручная подготовка отчетов по поставщикам выбранного материала требует дополнительных операций фильтрации и группировки.

Разрабатываемое приложение решает указанные проблемы за счет централизованного хранения данных и программной проверки операций. База данных контролирует существование материалов, поставщиков, типов документов и единиц измерения через внешние ключи. Серверная часть дополнительно проверяет бизнес-условия, например соответствие счета материала справочнику. Клиентская часть предоставляет формы, таблицы и отчеты, рассчитанные на повседневную работу сотрудника склада.

Основной информационный процесс системы начинается с авторизации пользователя, после чего приложение загружает справочники и список складских приходов. Пользователь может найти материал, добавить приходную операцию, получить количество поставщиков выбранного материала, открыть отчет по поставщикам и рассчитать количество поставщиков по банковскому адресу. Результатом является уменьшение ручных операций и повышение достоверности учетных данных.

1.2 Анализ аналогов разрабатываемого решения

Для автоматизации складского учета применяются как крупные корпоративные системы, так и облачные сервисы и табличные редакторы. Наиболее распространенными вариантами являются внедрение универсальной учетной системы, использование готового облачного сервиса либо ведение данных в электронных таблицах.

Система «1С:Предприятие» является промышленной платформой для управленческого, бухгалтерского и складского учета, применяется в организациях разного масштаба и имеет множество типовых конфигураций [7]. Ее преимуществами являются развитая функциональность, поддержка регламентированного учета и широкий рынок специалистов. Ограничением для учебного проекта являются избыточность, необходимость настройки конфигурации и зависимость от платформенной экосистемы.

Сервис «МойСклад» ориентирован на онлайн-управление торговлей, закупками, остатками, документами и поставщиками [8]. Он удобен для малого и среднего бизнеса, быстро запускается и уже содержит типовые бизнес-сценарии. Однако при использовании готового сервиса сложнее продемонстрировать собственное проектирование модели данных, серверной логики, процедур базы данных и клиентского интерфейса.

Табличные редакторы, например Microsoft Excel или Google Sheets, доступны и просты в освоении, но плохо подходят для контроля сложных связей между справочниками. В таблицах легко изменить ячейку без проверки

ссылочной целостности, ввести произвольную единицу измерения или нарушить правило расчета итоговой суммы. Кроме того, табличный подход затрудняет разграничение доступа и поддержку серверного API.

Таблица 1.1 - Сравнение вариантов решения задачи

Критерий	1С:Предприятие	МойСклад	Табличный редактор	Разрабатываемое приложение
Назначение	Комплексный учет предприятия	Облачный учет торговли и склада	Ручное хранение таблиц	Учебная система под заданную предметную модель
Гибкость модели данных	Высокая, но требует настройки	Ограничена возможностями сервиса	Формально свободная, но без строгих связей	Полностью соответствует заданию проекта
Контроль целостности	Реализуется платформой	Реализуется сервисом	Зависит от дисциплины пользователя	Внешние ключи, проверки, процедура и триггеры PostgreSQL
Демонстрация разработки	Скрыта за платформой	Скрыта за готовым сервисом	Не показывает серверную архитектуру	Показывает клиент, API, репозиторий и базу данных
Вывод	Подходит для промышленного внедрения	Подходит для быстрого запуска бизнеса	Подходит только для простого учета	Целесообразно для курсового проекта

Итогом анализа является вывод о необходимости разработки собственного решения. Такой вариант позволяет реализовать именно те процессы, которые требуются в проекте: авторизацию сотрудника, справочники материалов и поставщиков, добавление складского прихода, расчет итоговой стоимости, отчет по поставщикам материала и расчет количества поставщиков по банковскому адресу.

1.3 Описание требований к разрабатываемому решению

Требования к системе сформированы на основе анализа предметной области и возможностей реализованного программного продукта. Они разделены на функциональные, нефункциональные и требования к данным.

Таблица 1.2 - Требования к разрабатываемому решению

№	Требование	Описание реализации
1	Авторизация пользователя	Пользователь должен войти по почте и паролю либо зарегистрироваться по коду приглашения.
2	Просмотр материалов	Система должна отображать справочник материалов с кодом, классом, группой, названием и счетом.
3	Поиск материалов	Интерфейс должен поддерживать поиск по коду, названию, классу, группе и счету материала.
4	Просмотр поставщиков	Система должна хранить и отображать поставщиков с ИНН, юридическим адресом, банковским адресом и счетом.
5	Добавление единицы хранения	Пользователь должен ввести номер ордера, дату, поставщика, документ, материал, единицу измерения, количество и цену.
6	Расчет итоговой стоимости	Итоговая стоимость должна рассчитываться как произведение количества и цены единицы.
7	Проверка справочников	Система должна отклонять приход с неизвестным поставщиком, материалом, документом или единицей измерения.
8	Проверка счета материала	Счет материала в приходе должен соответствовать счету из справочника материалов.

9	Отчет по материалу	Система должна показывать список поставщиков выбранного материала, суммарное количество и сумму поставок.
10	Расчет по банковскому адресу	Система должна считать количество поставщиков, имеющих заданный банковский адрес.
11	Адаптивный интерфейс	Приложение должно корректно отображаться на экранах разной ширины.
12	Темная и светлая тема	Пользователь должен иметь возможность переключать визуальную тему интерфейса.
13	Разделение клиентской и серверной части	Клиент должен взаимодействовать с сервером через REST API.
14	Постоянное хранение данных	При наличии строки подключения данные должны храниться в PostgreSQL.

Нефункциональные требования включают понятность интерфейса, контролируемую обработку ошибок, расширяемость серверной архитектуры, переносимость клиентской части и возможность запуска проекта одной командой из корневой папки. Требования к данным предполагают уникальность кодов справочников, положительные значения количества и цены, а также наличие связей между приходами, материалами, поставщиками, типами документов и единицами измерения.

2 ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ

2.1 Выбор инструментов и/или средств для реализации проекта

В проекте использована клиент-серверная архитектура: браузерное приложение отвечает за пользовательский интерфейс, серверное API реализует обработку запросов, а база данных хранит справочники и приходные операции. Такой подход соответствует характеру предметной области, поскольку складские данные должны быть централизованы и доступны нескольким рабочим местам.

Для серверной части выбран ASP.NET Core. Данная платформа поддерживает создание Web API на основе контроллеров, маршрутизацию HTTP-запросов, middleware-конвейер, встроенные механизмы аутентификации и авторизации, а также генерацию OpenAPI-описания в режиме разработки [1]. В проекте сервер реализован на C# и .NET, что обеспечивает строгую типизацию моделей и удобную обработку бизнес-ошибок.

Для клиентской части выбран React. Компонентная модель React хорошо подходит для интерфейса, состоящего из форм, таблиц, отчетов, переключателей состояния и динамической загрузки данных. В приложении используются хуки `useState`, `useEffect` и `useMemo`, что соответствует официальным правилам разработки на React [2].

Для хранения данных выбрана PostgreSQL. СУБД поддерживает реляционную модель, внешние ключи, ограничения CHECK, вычисляемые столбцы, SQL-функции, процедуры и триггеры [3]. Эти возможности важны для складского учета, поскольку позволяют проверять связи между справочниками и приходными операциями на уровне базы данных.

Для сборки клиентского приложения используется Vite, так как он обеспечивает быстрый запуск dev-сервера и современную сборку frontend-проекта [4]. Для взаимодействия ASP.NET Core с PostgreSQL используется Npgsql, официальный провайдер доступа к PostgreSQL для .NET [5]. Типизация

клиентского кода выполнена с использованием TypeScript, что снижает риск ошибок при работе с объектами материалов, поставщиков и доходов [6].

Таблица 2.1 - Выбранные инструменты разработки

Компонент	Выбранное средство	Причина выбора
Серверная часть	ASP.NET Core, C#	Типизированные контроллеры, middleware, OpenAPI, удобная интеграция с авторизацией.
Клиентская часть	React, TypeScript	Компонентный интерфейс, хуки состояния, строгие типы для данных API.
Сборка клиента	Vite	Быстрый локальный запуск, современная сборка и поддержка React.
База данных	PostgreSQL	Связанные таблицы, ограничения, функции, процедура и триггеры.
Доступ к БД	Npgsql	Провайдер PostgreSQL для .NET и параметризованные SQL-команды.
Запуск проекта	PowerShell, npm, dotnet CLI	Единая команда запуска клиента и API из корневой папки.

2.2 Разработка проекта решения

Проект решения построен по слоистой схеме. Клиентская часть отображает интерфейс и отправляет HTTP-запросы. Серверная часть принимает запросы через контроллеры, проверяет авторизацию, передает операции в репозиторий и возвращает JSON-ответы. Репозиторий скрывает детали хранения данных и может работать либо с PostgreSQL, либо с резервной in-memory реализацией при отсутствии строки подключения.

Серверное приложение состоит из нескольких основных частей: Program.cs настраивает сервисы, CORS, OpenAPI, аутентификацию и выбор репозитория; AuthController обрабатывает вход, регистрацию и получение текущего пользователя; WarehouseController предоставляет операции

складского учета; модели из WarehouseModels.cs описывают материалы, поставщиков, адреса, документы, единицы измерения, приходы и ответы авторизации.

Основные маршруты API: GET /api/warehouse/materials, GET /api/warehouse/suppliers, GET /api/warehouse/documents, GET /api/warehouse/measurement-units, GET /api/warehouse/storage-units, POST /api/warehouse/storage-units, GET /api/warehouse/materials/{materialCode}/supplier-count, GET /api/warehouse/materials/{materialCode}/suppliers, POST /api/warehouse/suppliers/bank-address-count. Для доступа к складским маршрутам требуется токен авторизации.

Физическая структура базы данных включает таблицы suppliers, materials, measurement_units, document_types и storage_units. В таблице storage_units итоговая стоимость total_price является вычисляемым столбцом. Процедура add_storage_unit добавляет приход и проверяет соответствие счета материала справочнику. Функции count_suppliers_for_material, get_suppliers_for_material и count_suppliers_by_bank_address реализуют отчетные операции. Триггеры поддерживают каскадное изменение счета материала, удаление зависимых приходов при удалении единицы измерения и запрет удаления поставщика с приходами.



Рисунок 2.1 - Диаграмма состояний системы

Диаграмма состояний отражает основной жизненный цикл приложения: запуск, загрузку справочников, переход на рабочий экран, ввод складского прихода, проверку данных, обновление таблиц, просмотр отчетов и обработку ошибок.

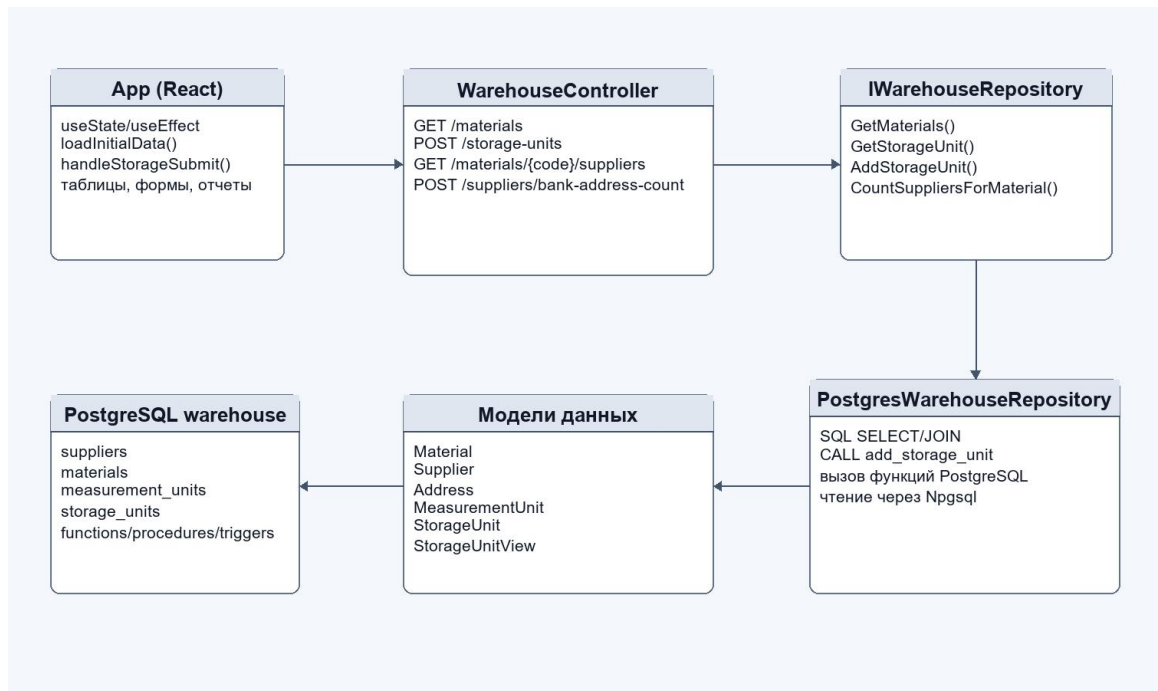


Рисунок 2.2 - Диаграмма классов и компонентов складского модуля

Диаграмма классов показывает взаимодействие React-приложения, контроллера WarehouseController, интерфейса IRepository, реализации PostgresWarehouseRepository, моделей данных и схемы PostgreSQL.

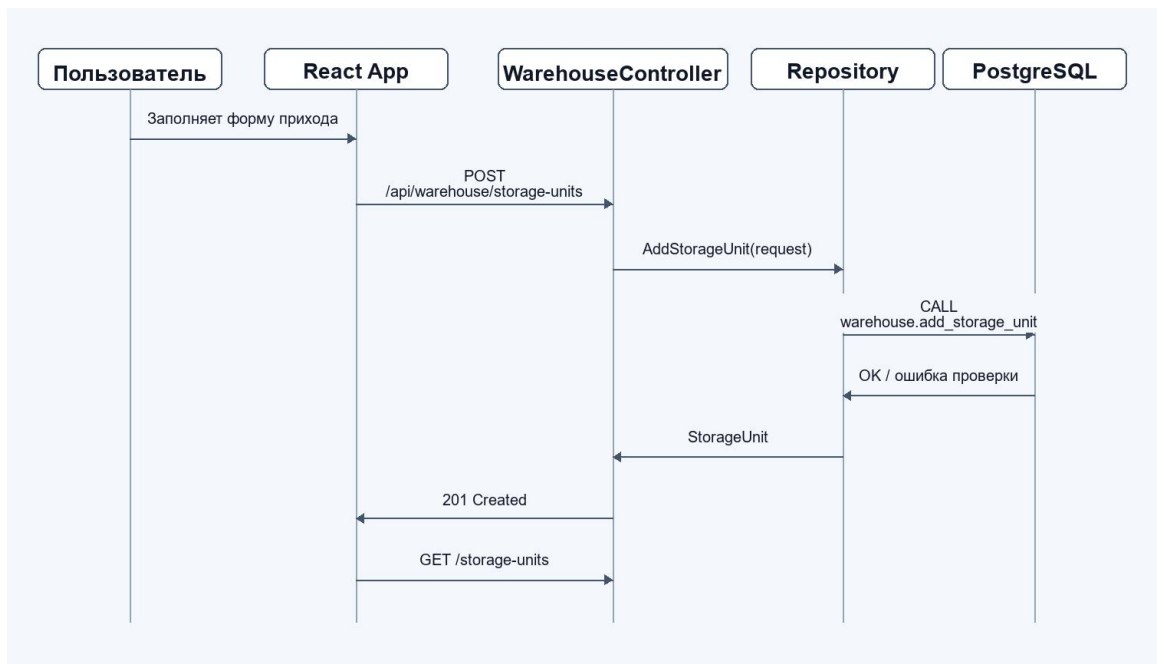


Рисунок 2.3 - Диаграмма последовательности добавления складского прихода

Диаграмма последовательности описывает типовой сценарий добавления единицы хранения. Пользователь заполняет форму, React-приложение отправляет POST-запрос, контроллер передает запрос в репозиторий, репозиторий вызывает процедуру PostgreSQL, база данных выполняет проверки и возвращает результат.

3 ПРАКТИЧЕСКАЯ ЧАСТЬ

3.1 Разработка составных элементов

Разработка программного продукта выполнена в виде двух приложений и набора SQL-объектов базы данных. Клиентская часть находится в каталоге `warehouse-client`, серверная часть - в каталоге `Warehouse.Api`, схема базы данных - в файле `database/warehouse.sql`. Корневая команда `npm run dev` запускает обе части проекта через сценарий `start.ps1`.

Клиентский компонент `App.tsx` содержит типы данных, состояние приложения, функции загрузки справочников, обработчики авторизации, добавления прихода и построения отчетов. Интерфейс состоит из экрана входа, панели показателей склада, блока материалов, формы добавления единицы хранения, таблицы приходов, отчета по поставщикам материала и формы расчета поставщиков по банковскому адресу.

Серверная часть реализует два контроллера. `AuthController` предоставляет вход, регистрацию по коду приглашения и получение текущего пользователя. `WarehouseController` предоставляет складские операции и защищен атрибутом `Authorize`. В `AuthService` реализовано хранение пользователей в памяти, хеширование паролей с солью, выдача токена и проверка токена в `TokenAuthenticationHandler`.

Репозиторий `IWarehouseRepository` задает контракт складских операций. In-memory реализация используется при пустой строке подключения и удобна для демонстрации без установленной СУБД. `PostgresWarehouseRepository` выполняет SQL-запросы через `Npgsql`, вызывает функции и процедуру PostgreSQL и преобразует строки результата в типизированные модели C#.

Схема PostgreSQL содержит справочники и таблицу приходных операций. Для исходной демонстрации добавлены три поставщика, три материала, несколько единиц измерения, типы документов и четыре прихода. Такая начальная база позволяет проверить работу справочников, формы добавления и отчетов сразу после запуска.

3.2 Установка и настройка

Для запуска проекта необходимы Node.js с npm и установленный .NET SDK, соответствующий целевой платформе проекта. При использовании PostgreSQL дополнительно создается база данных, в которой выполняется сценарий `database/warehouse.sql`. Если строка подключения `WarehouseDb` в `appsettings.json` оставлена пустой, сервер использует in-memory хранилище и позволяет запустить демонстрацию без отдельной СУБД.

Порядок запуска проекта:

- открыть корневую папку проекта;
- выполнить команду `npm run dev`;
- дождаться запуска API на `http://localhost:5064`;
- открыть клиентское приложение по адресу `http://localhost:5173`;
- войти под тестовой учетной записью `employee@warehouse.local` с паролем `Warehouse123!` либо зарегистрироваться по коду `INVITE-2026`.

Сценарий `start.ps1` проверяет наличие `npm` и `dotnet`, при необходимости устанавливает зависимости клиентской части, выполняет восстановление пакетов `backend`-проекта, останавливает ранее запущенные процессы этого приложения и стартует API и Vite-клиент в скрытых фоновых окнах. Логи запуска сохраняются в каталоге `.run`.

Основные параметры настройки хранятся в `appsettings.json`. В разделе `Auth` указаны демонстрационная учетная запись и допустимые коды приглашения. В разделе `ConnectionStrings` указывается строка подключения `WarehouseDb`. Такой подход отделяет настройки окружения от исходного кода и позволяет запускать проект в разных режимах.

3.3 Тестирование

Функциональное тестирование проводилось по основным пользовательским сценариям. Проверялись авторизация, регистрация, загрузка справочников, работа поиска, добавление прихода, обработка ошибок

валидации и построение отчетов. Отдельно проверялась сборка клиентской и серверной части стандартными средствами проекта.

Таблица 3.1 - Функциональные тест-кейсы

№	Сценарий	Входные данные	Ожидаемый результат
1	Вход пользователя	employee@warehouse.local, Warehouse123!	Пользователь попадает на рабочий экран склада.
2	Ошибочный вход	Неверный пароль	Система показывает сообщение об ошибке и не открывает складские данные.
3	Регистрация по приглашению	Корректная почта, имя, пароль и INVITE-2026	Создается учетная запись и выдается токен доступа.
4	Загрузка справочников	Открытие рабочего экрана	Отображаются материалы, поставщики, документы, единицы измерения и приходы.
5	Поиск материала	Фрагмент названия или кода материала	Таблица материалов показывает только подходящие строки.
6	Добавление прихода	Номер ордера, дата, поставщик, документ, материал, единица, количество и цена	Приход добавляется, таблица обновляется, итоговая стоимость рассчитана.
7	Проверка счета материала	Счет, не соответствующий выбранному материалу	Сервер возвращает ошибку, запись не добавляется.
8	Отчет по материалу	Выбор МАТ-001	Отображается количество поставщиков и список поставщиков с суммарными значениями.
9	Расчет по	101000, Москва,	Система возвращает

	банковскому адресу	Мясницкая, 12	количество поставщиков с указанным банковским адресом.
10	Выход пользователя	Нажатие кнопки «Выйти»	Токен удаляется, пользователь возвращается на экран входа.

По результатам тестирования основные требования были выполнены. Приложение корректно разделяет доступ к складским маршрутам, загружает справочники, выполняет валидацию данных, рассчитывает итоговые показатели и обновляет пользовательский интерфейс после операций.

3.4 Ввод в эксплуатацию

Ввод приложения в эксплуатацию предполагает подготовку серверного окружения, настройку базы данных и публикацию клиентской части. На этапе учебной демонстрации приложение запускается локально: API доступен на порту 5064, клиентское приложение - на порту 5173. Для промышленного режима необходимо разместить API на сервере, настроить HTTPS, указать постоянную строку подключения к PostgreSQL и собрать клиентскую часть командой `npm run build`.

Первоначальное наполнение базы выполняется SQL-сценарием `database/warehouse.sql`. После выполнения сценария создается схема `warehouse`, таблицы, функции, процедура, триггеры и демонстрационные записи. Далее администратор может заменить демонстрационные данные фактическими справочниками предприятия.

Для безопасной эксплуатации необходимо заменить демонстрационный пароль, задать собственные коды приглашения, ограничить CORS доменом реального клиентского приложения и хранить строку подключения вне публичного исходного кода, например через переменные окружения или секреты окружения развертывания.

3.5 Разработка сопроводительной документации

Сопроводительная документация включает инструкцию пользователя, описание настройки и перечень основных файлов проекта. Инструкция пользователя предназначена для сотрудника склада, который выполняет вход, просматривает материалы, добавляет приходные операции и формирует отчеты.

Инструкция по эксплуатации:

- запустить проект командой `npm run dev` из корневой папки;
- открыть в браузере `http://localhost:5173`;
- войти под существующей учетной записью или зарегистрироваться по коду приглашения;
- для просмотра материалов открыть блок «Последние материалы» и перейти ко всему справочнику;
- для добавления прихода заполнить форму «Добавление единицы хранения» и нажать кнопку «Добавить»;
- для просмотра отчета выбрать материал в блоке поставщиков материала;
- для расчета поставщиков по банковскому адресу заполнить адрес банка и выполнить расчет;
- для завершения работы нажать кнопку «Выйти».

Таблица 3.2 - Основные файлы проекта

Файл или каталог	Назначение
warehouse-client/src/App.tsx	Главный React-компонент, состояние интерфейса, формы, таблицы, авторизация и вызовы API.
warehouse-client/src/App.css	Стили интерфейса, адаптивная верстка, светлая и темная тема.
Warehouse.Api/Program.cs	Настройка серверного приложения, сервисов, CORS, аутентификации и маршрутизации.
Warehouse.Api/Controllers/AuthController.cs	Маршруты входа, регистрации и получения текущего пользователя.
Warehouse.Api/Controllers/WarehouseController.cs	Маршруты складского учета и отчетов.
Warehouse.Api/Services/AuthService.cs	Пользователи, хеширование паролей, токены и обработчик

	аутентификации.
Warehouse.Api/Services/WarehouseRepository.cs	Контракт репозитория и in-memory реализация для демонстрации.
Warehouse.Api/Services/PostgresWarehouseRepository.cs	Реализация репозитория для PostgreSQL через Npgsql.
Warehouse.Api/Models/WarehouseModels.cs	Типизированные модели предметной области и авторизации.
database/warehouse.sql	Создание схемы базы данных, функций, процедуры, триггеров и начальных данных.
start.ps1	Автоматический запуск backend и frontend частей проекта.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы была описана программируемая система складского учета материалов, рассмотрены существующие решения-аналоги по теме, сформированы требования к разрабатываемому решению, выбран технологический стек, спроектирована архитектура приложения и разработан программный продукт.

Практическая реализация включает React-клиент, ASP.NET Core Web API, модуль авторизации, репозиторий данных и схему PostgreSQL. Система позволяет выполнять вход и регистрацию пользователя, просматривать справочники материалов и поставщиков, добавлять складские приходы, рассчитывать итоговую стоимость, формировать отчет по поставщикам выбранного материала и определять количество поставщиков по банковскому адресу.

Поставленные задачи выполнены: предметная область проанализирована, аналоги рассмотрены, требования описаны, инструменты выбраны и обоснованы, проект решения представлен диаграммами и описанием структуры, основные составные элементы реализованы, подготовлена инструкция по установке, тестированию и эксплуатации.

Перспективы развития продукта могут состоять в добавлении ролей доступа, редактировании справочников через интерфейс, журналировании действий пользователей, экспорте отчетов в XLSX или PDF, подключении промышленной базы данных предприятия, расширении аналитики по остаткам и интеграции с внешними учетными системами.

Практическая и исследовательская значимость работы заключается не только в получении непосредственного опыта разработки веб-приложения, но и в изучении общего контекста процесса создания программного продукта: анализа требований, проектирования, выбора технологий, реализации, тестирования, настройки и подготовки сопроводительной документации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Microsoft Learn. Create web APIs with ASP.NET Core [Электронный ресурс]. URL: <https://learn.microsoft.com/en-us/aspnet/core/web-api/?view=aspnetcore-10.0> (дата обращения: 19.05.2026).
2. React Documentation. React Reference Overview [Электронный ресурс]. URL: <https://react.dev/reference/react> (дата обращения: 19.05.2026).
3. PostgreSQL Documentation. PostgreSQL 18 Documentation [Электронный ресурс]. URL: <https://www.postgresql.org/docs/current/> (дата обращения: 19.05.2026).
4. Vite Documentation. Getting Started [Электронный ресурс]. URL: <https://vite.dev/guide/> (дата обращения: 19.05.2026).
5. Npgsql Documentation. Basic Usage [Электронный ресурс]. URL: <https://www.npgsql.org/doc/basic-usage.html> (дата обращения: 19.05.2026).
6. TypeScript Documentation [Электронный ресурс]. URL: <https://www.typescriptlang.org/docs/> (дата обращения: 19.05.2026).
7. Система программ 1С:Предприятие, официальный сайт [Электронный ресурс]. URL: <https://v8.1c.ru/> (дата обращения: 19.05.2026).
8. МойСклад. Облачная ERP-система [Электронный ресурс]. URL: <https://www.moysklad.ru/> (дата обращения: 19.05.2026).

ПРИЛОЖЕНИЯ

Приложение А - Исходный код серверной части

В приложении приведены ключевые файлы серверного приложения ASP.NET Core.

Листинг А.1 - Program.cs

```
using Npgsql;
using Warehouse.Api.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddOpenApi();
builder.Services.Configure<AuthOptions>(builder.Configuration.GetSection("Auth"));
builder.Services.AddSingleton<IAuthService, AuthService>();
builder.Services
    .AddAuthentication("WarehouseToken")
    .AddScheme<Microsoft.AspNetCore.Authentication.AuthenticationSchemeOptions, TokenAuthenticationHandler>(
        "WarehouseToken",
        _ => { });
builder.Services.AddAuthorization();

var warehouseConnectionString = builder.Configuration.GetConnectionString("WarehouseDb");
if (string.IsNullOrEmpty(warehouseConnectionString))
{
    builder.Services.AddSingleton<IWarehouseRepository, WarehouseRepository>();
}
else
{
    builder.Services.AddSingleton(NpgsqlDataSource.Create(warehouseConnectionString));
    builder.Services.AddSingleton<IWarehouseRepository, PostgresWarehouseRepository>();
}

builder.Services.AddCors(options =>
{
    options.AddPolicy("ReactClient", policy =>
        policy.WithOrigins("http://localhost:5173", "https://localhost:5173")
            .AllowAnyHeader()
            .AllowAnyMethod());
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

app.UseHttpsRedirection();

app.UseCors("ReactClient");

app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();

app.Run();
```

Листинг А.2 - AuthController.cs

```

using System.Security.Claims;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Warehouse.Api.Models;
using Warehouse.Api.Services;

namespace Warehouse.Api.Controllers;

[ApiController]
[Route("api/auth")]
public sealed class AuthController(IAuthService authService) : ControllerBase
{
    [HttpPost("login")]
    public IActionResult Login(LoginRequest request)
    {
        try
        {
            return Ok(authService.Login(request));
        }
        catch (InvalidOperationException exception)
        {
            return BadRequest(new { message = exception.Message });
        }
    }

    [HttpPost("register")]
    public IActionResult Register(RegisterRequest request)
    {
        try
        {
            return Ok(authService.Register(request));
        }
        catch (InvalidOperationException exception)
        {
            return BadRequest(new { message = exception.Message });
        }
    }

    [Authorize]
    [HttpGet("me")]
    public IActionResult Me()
    {
        return Ok(new AuthUser(
            User.FindFirstValue(ClaimTypes.Email) ?? string.Empty,
            User.FindFirstValue(ClaimTypes.Name) ?? string.Empty));
    }
}

```

Листинг A.3 - WarehouseController.cs

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Warehouse.Api.Models;
using Warehouse.Api.Services;

namespace Warehouse.Api.Controllers;

[ApiController]
[Authorize]
[Route("api/warehouse")]
public sealed class WarehouseController(IWarehouseRepository repository) : ControllerBase
{
    [HttpGet("materials")]
    public IActionResult GetMaterials() => Ok(repository.GetMaterials());

    [HttpGet("suppliers")]
    public IActionResult GetSuppliers() => Ok(repository.GetSuppliers());

    [HttpGet("documents")]
    public IActionResult GetDocumentTypes() => Ok(repository.GetDocumentTypes());
}

```

```

[HttpGet("measurement-units")]
public IActionResult GetMeasurementUnits([FromQuery] string? materialCode = null) =>
    Ok(repository.GetMeasurementUnits(materialCode));

[HttpGet("storage-units")]
public IActionResult GetStorageUnits() => Ok(repository.GetStorageUnits());

[HttpPost("storage-units")]
public IActionResult AddStorageUnit(StorageUnitCreateRequest request)
{
    try
    {
        return CreatedAtAction(nameof(GetStorageUnits), repository.AddStorageUnit(request));
    }
    catch (InvalidOperationException exception)
    {
        return BadRequest(new { message = exception.Message });
    }
}

[HttpGet("materials/{materialCode}/supplier-count")]
public IActionResult CountSuppliersForMaterial(string materialCode) =>
    Ok(new { materialCode, count = repository.CountSuppliersForMaterial(materialCode) });

[HttpGet("materials/{materialCode}/suppliers")]
public IActionResult GetSuppliersForMaterial(string materialCode)
{
    try
    {
        return Ok(repository.GetSuppliersForMaterial(materialCode));
    }
    catch (InvalidOperationException exception)
    {
        return NotFound(new { message = exception.Message });
    }
}

[HttpPost("suppliers/bank-address-count")]
public IActionResult CountSuppliersByBankAddress(Address bankAddress) =>
    Ok(new { count = repository.CountSuppliersByBankAddress(bankAddress) });
}

```

Листинг А.4 - WarehouseModels.cs

```

namespace Warehouse.Api.Models;

public sealed record Material(
    string Code,
    string ClassCode,
    string GroupCode,
    string Name,
    string MaterialAccount);

public sealed record MeasurementUnit(
    string MaterialCode,
    string UnitCode,
    string UnitName);

public sealed record Supplier(
    string Code,
    string Name,
    string Inn,
    Address LegalAddress,
    Address BankAddress,
    string BankAccountNumber);

public sealed record Address(
    string PostalCode,
    string City,
    string Street,
    string House);

```

```

public sealed record DocumentType(
    string Code,
    string Name);

public sealed record StorageUnit(
    int OrderNumber,
    DateOnly OrderDate,
    string SupplierCode,
    string BalanceAccount,
    string DocumentTypeCode,
    string DocumentNumber,
    string MaterialCode,
    string MaterialAccount,
    string UnitCode,
    decimal Quantity,
    decimal UnitPrice);

public sealed record StorageUnitCreateRequest(
    int OrderNumber,
    DateOnly OrderDate,
    string SupplierCode,
    string BalanceAccount,
    string DocumentTypeCode,
    string DocumentNumber,
    string MaterialCode,
    string MaterialAccount,
    string UnitCode,
    decimal Quantity,
    decimal UnitPrice);

public sealed record StorageUnitView(
    int OrderNumber,
    DateOnly OrderDate,
    string SupplierName,
    string MaterialName,
    string UnitName,
    decimal Quantity,
    decimal UnitPrice,
    decimal TotalPrice,
    string DocumentNumber);

public sealed record SupplierForMaterialView(
    string MaterialCode,
    string MaterialName,
    string SupplierCode,
    string SupplierName,
    string Inn,
    Address LegalAddress,
    Address BankAddress,
    string BankAccountNumber,
    decimal TotalQuantity,
    decimal TotalAmount);

public sealed record AuthUser(
    string Email,
    string Name);

public sealed record LoginRequest(
    string Email,
    string Password);

public sealed record RegisterRequest(
    string Email,
    string Name,
    string Password,
    string InvitationCode);

public sealed record AuthResponse(
    string Token,
    AuthUser User);

```

Листинг A.5 - AuthService.cs

```
using System.Collections.Concurrent;
using System.Security.Claims;
using System.Security.Cryptography;
using System.Text;
using System.Text.Encodings.Web;
using Microsoft.AspNetCore.Authentication;
using Microsoft.Extensions.Options;
using Warehouse.Api.Models;

namespace Warehouse.Api.Services;

public sealed class AuthOptions
{
    public string SeedUserEmail { get; set; } = "employee@warehouse.local";
    public string SeedUserName { get; set; } = "Сотрудник склада";
    public string SeedUserPassword { get; set; } = "Warehouse123!";
    public string[] InvitationCodes { get; set; } = ["INVITE-2026", "WAREHOUSE-ACCESS"];
}

public interface IAuthService
{
    AuthResponse Login(LoginRequest request);
    AuthResponse Register(RegisterRequest request);
    AuthUser? ValidateToken(string token);
}

public sealed class AuthService : IAuthService
{
    private readonly ConcurrentDictionary<string, UserAccount> _users = new(StringComparer.OrdinalIgnoreCase);
    private readonly ConcurrentDictionary<string, string> _tokens = new();
    private readonly HashSet<string> _invitationCodes;

    public AuthService(IOptions<AuthOptions> options)
    {
        var authOptions = options.Value;
        _invitationCodes = authOptions.InvitationCodes
            .Where(code => !string.IsNullOrEmpty(code))
            .Select(code => code.Trim())
            .ToHashSet(StringComparer.OrdinalIgnoreCase);

        AddUser(authOptions.SeedUserEmail, authOptions.SeedUserName, authOptions.SeedUserPassword);
    }

    public AuthResponse Login(LoginRequest request)
    {
        if (!_users.TryGetValue(NormalizeEmail(request.Email), out var user)
            || !VerifyPassword(request.Password, user.PasswordHash, user.PasswordSalt))
        {
            throw new InvalidOperationException("Неверная почта или пароль.");
        }

        return IssueToken(user);
    }

    public AuthResponse Register(RegisterRequest request)
    {
        if (!_invitationCodes.Contains(request.InvitationCode.Trim()))
        {
            throw new InvalidOperationException("Код приглашения недействителен.");
        }

        if (string.IsNullOrEmpty(request.Name))
        {
            throw new InvalidOperationException("Укажите имя пользователя.");
        }

        if (request.Password.Length < 6)
```

```

    {
        throw new InvalidOperationException("Пароль должен быть не короче 6 символов.");
    }

    var user = CreateUser(request.Email, request.Name.Trim(), request.Password);

    if (!_users.TryAdd(user.Email, user))
    {
        throw new InvalidOperationException("Пользователь с такой почтой уже зарегистрирован.");
    }

    return IssueToken(user);
}

public AuthUser? ValidateToken(string token)
{
    if (!_tokens.TryGetValue(token, out var email) || !_users.TryGetValue(email, out var user))
    {
        return null;
    }

    return new AuthUser(user.Email, user.Name);
}

private void AddUser(string email, string name, string password)
{
    var user = CreateUser(email, name, password);
    _users.TryAdd(user.Email, user);
}

private static UserAccount CreateUser(string email, string name, string password)
{
    var normalizedEmail = NormalizeEmail(email);

    if (string.IsNullOrEmpty(normalizedEmail) || !normalizedEmail.Contains('@'))
    {
        throw new InvalidOperationException("Укажите корректную почту.");
    }

    var salt = RandomNumberGenerator.GetBytes(16);
    return new UserAccount(normalizedEmail, name, HashPassword(password, salt), salt);
}

private AuthResponse IssueToken(UserAccount user)
{
    var token = Convert.ToBase64String(RandomNumberGenerator.GetBytes(32));
    _tokens[token] = user.Email;
    return new AuthResponse(token, new AuthUser(user.Email, user.Name));
}

private static string NormalizeEmail(string email) => email.Trim().ToLowerInvariant();

private static byte[] HashPassword(string password, byte[] salt) =>
    Rfc2898DeriveBytes.Pbkdf2(
        Encoding.UTF8.GetBytes(password),
        salt,
        100_000,
        HashAlgorithmName.SHA256,
        32);

private static bool VerifyPassword(string password, byte[] hash, byte[] salt)
{
    var candidate = HashPassword(password, salt);
    return CryptographicOperations.FixedTimeEquals(candidate, hash);
}

private sealed record UserAccount(

```

```

        string Email,
        string Name,
        byte[] PasswordHash,
        byte[] PasswordSalt);
    }

    public sealed class TokenAuthenticationHandler(
        IOptionsMonitor<AuthenticationSchemeOptions> options,
        ILoggerFactory logger,
        UriEncoder encoder,
        IAuthService authService)
        : AuthenticationHandler<AuthenticationSchemeOptions>(options, logger, encoder)
    {
        protected override Task<AuthenticateResult> HandleAuthenticateAsync()
        {
            var header = Request.Headers.Authorization.ToString();

            if (!header.StartsWith("Bearer ", StringComparison.OrdinalIgnoreCase))
            {
                return Task.FromResult(AuthenticateResult.NoResult());
            }

            var token = header["Bearer ".Length..].Trim();
            var user = authService.ValidateToken(token);

            if (user is null)
            {
                return Task.FromResult(AuthenticateResult.Fail("Invalid token."));
            }

            var claims = new[]
            {
                new Claim(ClaimTypes.NameIdentifier, user.Email),
                new Claim(ClaimTypes.Name, user.Name),
                new Claim(ClaimTypes.Email, user.Email),
            };
            var identity = new ClaimsIdentity(claims, Scheme.Name);
            var principal = new ClaimsPrincipal(identity);
            var ticket = new AuthenticationTicket(principal, Scheme.Name);

            return Task.FromResult(AuthenticateResult.Success(ticket));
        }
    }
}

```

Листинг A.6 - WarehouseRepository.cs

```

using Warehouse.Api.Models;

namespace Warehouse.Api.Services;

public interface IWarehouseRepository
{
    IReadOnlyCollection<Material> GetMaterials();
    IReadOnlyCollection<Supplier> GetSuppliers();
    IReadOnlyCollection<DocumentType> GetDocumentTypes();
    IReadOnlyCollection<MeasurementUnit> GetMeasurementUnits(string? materialCode = null);
    IReadOnlyCollection<StorageUnitView> GetStorageUnits();
    StorageUnit AddStorageUnit(StorageUnitCreateRequest request);
    int CountSuppliersForMaterial(string materialCode);
    IReadOnlyCollection<SupplierForMaterialView> GetSuppliersForMaterial(string materialCode);
    int CountSuppliersByBankAddress(Address bankAddress);
}

public sealed class WarehouseRepository : IWarehouseRepository
{
    private readonly List<Material> _materials =
    [
        new("MAT-001", "10", "101", "Лист стальной 2 мм", "10.01"),
        new("MAT-002", "20", "204", "Краска акриловая белая", "10.06"),
        new("MAT-003", "30", "302", "Кабель силовой ВВГнг", "10.05")
    ];
}

```

```

private readonly List<MeasurementUnit> _measurementUnits =
[
    new("MAT-001", "KG", "килограммы"),
    new("MAT-001", "PCS", "штуки"),
    new("MAT-002", "L", "литры"),
    new("MAT-003", "M", "метры")
];

private readonly List<Supplier> _suppliers =
[
    new(
        "SUP-001",
        "ООО МеталлСнаб",
        "7708123456",
        new Address("125047", "Москва", "Лесная", "5"),
        new Address("101000", "Москва", "Мясницкая", "12"),
        "4070281090000000001"),
    new(
        "SUP-002",
        "АО ПромКомплект",
        "7816123001",
        new Address("190000", "Санкт-Петербург", "Садовая", "21"),
        new Address("101000", "Москва", "Мясницкая", "12"),
        "4070281090000000002"),
    new(
        "SUP-003",
        "ИП Волкова Н.А.",
        "502900771802",
        new Address("141008", "Мытищи", "Мира", "18"),
        new Address("141002", "Мытищи", "Новомиיתיшинский", "31"),
        "4080281040000000003")
];

private readonly List<DocumentType> _documentTypes =
[
    new("INV", "Счет-фактура"),
    new("WAY", "Товарная накладная"),
    new("ACT", "Акт приемки")
];

private readonly List<StorageUnit> _storageUnits =
[
    new(1001, new DateOnly(2026, 4, 8), "SUP-001", "15", "WAY", "TH-450", "MAT-001", "10.01", "KG", 1200,
88.50m),
    new(1002, new DateOnly(2026, 4, 12), "SUP-002", "15", "INV", "СФ-981", "MAT-001", "10.01", "PCS", 350,
145.00m),
    new(1003, new DateOnly(2026, 4, 19), "SUP-003", "15", "WAY", "TH-118", "MAT-002", "10.06", "L", 90,
520.00m),
    new(1004, new DateOnly(2026, 4, 28), "SUP-002", "15", "ACT", "АП-77", "MAT-003", "10.05", "M", 800,
74.30m)
];

private readonly object _sync = new();

public IReadOnlyCollection<Material> GetMaterials() => _materials;

public IReadOnlyCollection<Supplier> GetSuppliers() => _suppliers;

public IReadOnlyCollection<DocumentType> GetDocumentTypes() => _documentTypes;

public IReadOnlyCollection<MeasurementUnit> GetMeasurementUnits(string? materialCode = null)
{
    return string.IsNullOrEmpty(materialCode)
        ? _measurementUnits
        : _measurementUnits
            .Where(unit => unit.MaterialCode.Equals(materialCode, StringComparison.OrdinalIgnoreCase))
            .ToArray();
}

```

```

public IReadOnlyCollection<StorageUnitView> GetStorageUnits()
{
    lock (_sync)
    {
        return _storageUnits
            .OrderByDescending(unit => unit.OrderDate)
            .ThenByDescending(unit => unit.OrderNumber)
            .Select(ToView)
            .ToArray();
    }
}

public StorageUnit AddStorageUnit(StorageUnitCreateRequest request)
{
    ValidateRequest(request);

    var storageUnit = new StorageUnit(
        request.OrderNumber,
        request.OrderDate,
        request.SupplierCode,
        request.BalanceAccount,
        request.DocumentTypeCode,
        request.DocumentNumber,
        request.MaterialCode,
        request.MaterialAccount,
        request.UnitCode,
        request.Quantity,
        request.UnitPrice);

    lock (_sync)
    {
        if (_storageUnits.Any(unit => unit.OrderNumber == request.OrderNumber))
        {
            throw new InvalidOperationException("Единица хранения с таким номером ордера уже существует.");
        }

        _storageUnits.Add(storageUnit);
    }

    return storageUnit;
}

public int CountSuppliersForMaterial(string materialCode)
{
    return _storageUnits
        .Where(unit => unit.MaterialCode.Equals(materialCode, StringComparison.OrdinalIgnoreCase))
        .Select(unit => unit.SupplierCode)
        .Distinct(StringComparer.OrdinalIgnoreCase)
        .Count();
}

public IReadOnlyCollection<SupplierForMaterialView> GetSuppliersForMaterial(string materialCode)
{
    var material = FindMaterial(materialCode);

    return _storageUnits
        .Where(unit => unit.MaterialCode.Equals(materialCode, StringComparison.OrdinalIgnoreCase))
        .GroupBy(unit => unit.SupplierCode, StringComparer.OrdinalIgnoreCase)
        .Select(group =>
        {
            var supplier = FindSupplier(group.Key);
            return new SupplierForMaterialView(
                material.Code,
                material.Name,
                supplier.Code,
                supplier.Name,
                supplier.Inn,
                supplier.LegalAddress,
                supplier.BankAddress,
                supplier.BankAccountNumber,
                group.Sum(unit => unit.Quantity),
            );
        });
}

```

```

        group.Sum(unit => unit.Quantity * unit.UnitPrice));
    })
    .OrderBy(item => item.SupplierName)
    .ToArray();
}

public int CountSuppliersByBankAddress(Address bankAddress)
{
    return _suppliers.Count(supplier =>
        SameAddress(supplier.BankAddress, bankAddress));
}

private StorageUnitView ToView(StorageUnit unit)
{
    var supplier = FindSupplier(unit.SupplierCode);
    var material = FindMaterial(unit.MaterialCode);
    var measurementUnit = _measurementUnits.First(item =>
        item.MaterialCode == unit.MaterialCode && item.UnitCode == unit.UnitCode);

    return new StorageUnitView(
        unit.OrderNumber,
        unit.OrderDate,
        supplier.Name,
        material.Name,
        measurementUnit.UnitName,
        unit.Quantity,
        unit.UnitPrice,
        unit.Quantity * unit.UnitPrice,
        unit.DocumentNumber);
}

private void ValidateRequest(StorageUnitCreateRequest request)
{
    _ = FindSupplier(request.SupplierCode);
    var material = FindMaterial(request.MaterialCode);

    if (!_documentTypes.Any(document => document.Code == request.DocumentTypeCode))
    {
        throw new InvalidOperationException("Неизвестный код сопроводительного документа.");
    }

    if (!_measurementUnits.Any(unit =>
        unit.MaterialCode == request.MaterialCode && unit.UnitCode == request.UnitCode))
    {
        throw new InvalidOperationException("Для выбранного материала нет такой единицы измерения.");
    }

    if (request.Quantity <= 0 || request.UnitPrice <= 0)
    {
        throw new InvalidOperationException("Количество и цена должны быть больше нуля.");
    }

    if (!string.Equals(material.MaterialAccount, request.MaterialAccount,
        StringComparison.OrdinalIgnoreCase))
    {
        throw new InvalidOperationException("Счет материала не соответствует справочнику материалов.");
    }
}

private Supplier FindSupplier(string code) =>
    _suppliers.FirstOrDefault(supplier => supplier.Code.Equals(code, StringComparison.OrdinalIgnoreCase))
    ?? throw new InvalidOperationException("Поставщик не найден.");

private Material FindMaterial(string code) =>
    _materials.FirstOrDefault(material => material.Code.Equals(code, StringComparison.OrdinalIgnoreCase))
    ?? throw new InvalidOperationException("Материал не найден.");

private static bool SameAddress(Address left, Address right) =>
    Same(left.PostalCode, right.PostalCode)

```

```

        && Same(left.City, right.City)
        && Same(left.Street, right.Street)
        && Same(left.House, right.House);

private static bool Same(string left, string right) =>
    string.Equals(left.Trim(), right.Trim(), StringComparison.OrdinalIgnoreCase);
}

```

Листинг A.7 - PostgresWarehouseRepository.cs

```

using Npgsql;
using Warehouse.Api.Models;

namespace Warehouse.Api.Services;

public sealed class PostgresWarehouseRepository(NpgsqlDataSource dataSource) : IWarehouseRepository
{
    public IReadOnlyCollection<Material> GetMaterials()
    {
        using var command = dataSource.CreateCommand("""
            SELECT material_code, material_class_code, material_group_code, material_name, material_account
            FROM warehouse.materials
            ORDER BY material_name;
            """);

        using var reader = command.ExecuteReader();
        var items = new List<Material>();

        while (reader.Read())
        {
            items.Add(new Material(
                reader.GetString(0),
                reader.GetString(1),
                reader.GetString(2),
                reader.GetString(3),
                reader.GetString(4)));
        }

        return items;
    }

    public IReadOnlyCollection<Supplier> GetSuppliers()
    {
        using var command = dataSource.CreateCommand("""
            SELECT supplier_code, supplier_name, inn,
                legal_postal_code, legal_city, legal_street, legal_house,
                bank_postal_code, bank_city, bank_street, bank_house,
                bank_account_number
            FROM warehouse.suppliers
            ORDER BY supplier_name;
            """);

        using var reader = command.ExecuteReader();
        var items = new List<Supplier>();

        while (reader.Read())
        {
            items.Add(ReadSupplier(reader));
        }

        return items;
    }

    public IReadOnlyCollection<DocumentType> GetDocumentTypes()
    {
        using var command = dataSource.CreateCommand("""
            SELECT document_type_code, document_type_name
            FROM warehouse.document_types
            ORDER BY document_type_name;
            """);
    }
}

```

```

        using var reader = command.ExecuteReader();
        var items = new List<DocumentType>();

        while (reader.Read())
        {
            items.Add(new DocumentType(reader.GetString(0), reader.GetString(1)));
        }

        return items;
    }

    public IReadOnlyCollection<MeasurementUnit> GetMeasurementUnits(string? materialCode = null)
    {
        using var command = dataSource.CreateCommand("""
            SELECT material_code, unit_code, unit_name
            FROM warehouse.measurement_units
            WHERE @material_code IS NULL OR material_code = @material_code
            ORDER BY unit_name;
            """);
        command.Parameters.AddWithValue("material_code", (object?)materialCode ?? DBNull.Value);

        using var reader = command.ExecuteReader();
        var items = new List<MeasurementUnit>();

        while (reader.Read())
        {
            items.Add(new MeasurementUnit(reader.GetString(0), reader.GetString(1), reader.GetString(2)));
        }

        return items;
    }

    public IReadOnlyCollection<StorageUnitView> GetStorageUnits()
    {
        using var command = dataSource.CreateCommand("""
            SELECT u.order_number, u.order_date, s.supplier_name, m.material_name, mu.unit_name,
                   u.quantity, u.unit_price, u.total_price, u.document_number
            FROM warehouse.storage_units AS u
            JOIN warehouse.suppliers AS s ON s.supplier_code = u.supplier_code
            JOIN warehouse.materials AS m ON m.material_code = u.material_code
            JOIN warehouse.measurement_units AS mu
              ON mu.material_code = u.material_code
            AND mu.unit_code = u.unit_code
            ORDER BY u.order_date DESC, u.order_number DESC;
            """);

        using var reader = command.ExecuteReader();
        var items = new List<StorageUnitView>();

        while (reader.Read())
        {
            items.Add(new StorageUnitView(
                reader.GetInt32(0),
                reader.GetFieldValue<DateOnly>(1),
                reader.GetString(2),
                reader.GetString(3),
                reader.GetString(4),
                reader.GetDecimal(5),
                reader.GetDecimal(6),
                reader.GetDecimal(7),
                reader.GetString(8)));
        }

        return items;
    }

    public StorageUnit AddStorageUnit(StorageUnitCreateRequest request)
    {

```

```

using var command = dataSource.CreateCommand("""
    CALL warehouse.add_storage_unit(
        @order_number,
        @order_date,
        @supplier_code,
        @balance_account,
        @document_type_code,
        @document_number,
        @material_code,
        @material_account,
        @unit_code,
        @quantity,
        @unit_price
    );
""");

command.Parameters.AddWithValue("order_number", request.OrderNumber);
command.Parameters.AddWithValue("order_date", request.OrderDate);
command.Parameters.AddWithValue("supplier_code", request.SupplierCode);
command.Parameters.AddWithValue("balance_account", request.BalanceAccount);
command.Parameters.AddWithValue("document_type_code", request.DocumentTypeCode);
command.Parameters.AddWithValue("document_number", request.DocumentNumber);
command.Parameters.AddWithValue("material_code", request.MaterialCode);
command.Parameters.AddWithValue("material_account", request.MaterialAccount);
command.Parameters.AddWithValue("unit_code", request.UnitCode);
command.Parameters.AddWithValue("quantity", request.Quantity);
command.Parameters.AddWithValue("unit_price", request.UnitPrice);

try
{
    command.ExecuteNonQuery();
}
catch (PostgresException exception)
{
    throw new InvalidOperationException(exception.MessageText, exception);
}

return new StorageUnit(
    request.OrderNumber,
    request.OrderDate,
    request.SupplierCode,
    request.BalanceAccount,
    request.DocumentTypeCode,
    request.DocumentNumber,
    request.MaterialCode,
    request.MaterialAccount,
    request.UnitCode,
    request.Quantity,
    request.UnitPrice);
}

public int CountSuppliersForMaterial(string materialCode)
{
    using var command = dataSource.CreateCommand("SELECT
warehouse.count_suppliers_for_material(@material_code);");
    command.Parameters.AddWithValue("material_code", materialCode);
    return Convert.ToInt32(command.ExecuteScalar());
}

public IReadOnlyCollection<SupplierForMaterialView> GetSuppliersForMaterial(string materialCode)
{
    using var command = dataSource.CreateCommand("SELECT * FROM
warehouse.get_suppliers_for_material(@material_code);");
    command.Parameters.AddWithValue("material_code", materialCode);

    using var reader = command.ExecuteReader();
    var items = new List<SupplierForMaterialView>();

    while (reader.Read())
    {
        items.Add(new SupplierForMaterialView(
            reader.GetString(0),

```

```

        reader.GetString(1),
        reader.GetString(2),
        reader.GetString(3),
        reader.GetString(4),
        new Address(reader.GetString(5), reader.GetString(6), reader.GetString(7),
reader.GetString(8)),
        new Address(reader.GetString(9), reader.GetString(10), reader.GetString(11),
reader.GetString(12)),
        reader.GetString(13),
        reader.GetDecimal(14),
        reader.GetDecimal(15)));
    }

    return items;
}

public int CountSuppliersByBankAddress(Address bankAddress)
{
    using var command = dataSource.CreateCommand("""
        SELECT warehouse.count_suppliers_by_bank_address(
            @bank_postal_code,
            @bank_city,
            @bank_street,
            @bank_house
        );
        """);
    command.Parameters.AddWithValue("bank_postal_code", bankAddress.PostalCode);
    command.Parameters.AddWithValue("bank_city", bankAddress.City);
    command.Parameters.AddWithValue("bank_street", bankAddress.Street);
    command.Parameters.AddWithValue("bank_house", bankAddress.House);

    return Convert.ToInt32(command.ExecuteScalar());
}

private static Supplier ReadSupplier(NpgsqlDataReader reader) =>
    new(
        reader.GetString(0),
        reader.GetString(1),
        reader.GetString(2),
        new Address(reader.GetString(3), reader.GetString(4), reader.GetString(5), reader.GetString(6)),
        new Address(reader.GetString(7), reader.GetString(8), reader.GetString(9), reader.GetString(10)),
        reader.GetString(11));
}

```

Приложение Б - SQL-сценарий базы данных

Листинг Б.1 - warehouse.sql

\encoding UTF8

```

DROP SCHEMA IF EXISTS warehouse CASCADE;
CREATE SCHEMA warehouse;
SET search_path TO warehouse;

CREATE TABLE suppliers
(
    supplier_code varchar(20) PRIMARY KEY,
    supplier_name varchar(200) NOT NULL,
    inn varchar(12) NOT NULL UNIQUE,
    legal_postal_code varchar(10) NOT NULL,
    legal_city varchar(100) NOT NULL,
    legal_street varchar(120) NOT NULL,
    legal_house varchar(20) NOT NULL,
    bank_postal_code varchar(10) NOT NULL,
    bank_city varchar(100) NOT NULL,
    bank_street varchar(120) NOT NULL,
    bank_house varchar(20) NOT NULL,
    bank_account_number varchar(30) NOT NULL
);

```

```

CREATE TABLE materials
(
    material_code varchar(20) PRIMARY KEY,
    material_class_code varchar(20) NOT NULL,
    material_group_code varchar(20) NOT NULL,
    material_name varchar(200) NOT NULL,
    material_account varchar(30) NOT NULL
);

CREATE TABLE measurement_units
(
    material_code varchar(20) NOT NULL,
    unit_code varchar(20) NOT NULL,
    unit_name varchar(80) NOT NULL,
    PRIMARY KEY (material_code, unit_code),
    CONSTRAINT fk_measurement_units_materials
        FOREIGN KEY (material_code)
        REFERENCES materials(material_code)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

CREATE TABLE document_types
(
    document_type_code varchar(20) PRIMARY KEY,
    document_type_name varchar(120) NOT NULL
);

CREATE TABLE storage_units
(
    order_number integer PRIMARY KEY,
    order_date date NOT NULL,
    supplier_code varchar(20) NOT NULL,
    balance_account varchar(30) NOT NULL,
    document_type_code varchar(20) NOT NULL,
    document_number varchar(50) NOT NULL,
    material_code varchar(20) NOT NULL,
    material_account varchar(30) NOT NULL,
    unit_code varchar(20) NOT NULL,
    quantity numeric(18, 3) NOT NULL CHECK (quantity > 0),
    unit_price numeric(18, 2) NOT NULL CHECK (unit_price > 0),
    total_price numeric(18, 2) GENERATED ALWAYS AS (quantity * unit_price) STORED,
    CONSTRAINT fk_storage_units_suppliers
        FOREIGN KEY (supplier_code)
        REFERENCES suppliers(supplier_code)
        ON UPDATE CASCADE,
    CONSTRAINT fk_storage_units_document_types
        FOREIGN KEY (document_type_code)
        REFERENCES document_types(document_type_code)
        ON UPDATE CASCADE,
    CONSTRAINT fk_storage_units_materials
        FOREIGN KEY (material_code)
        REFERENCES materials(material_code)
        ON UPDATE CASCADE,
    CONSTRAINT fk_storage_units_measurement_units
        FOREIGN KEY (material_code, unit_code)
        REFERENCES measurement_units(material_code, unit_code)
);

INSERT INTO suppliers
(supplier_code, supplier_name, inn, legal_postal_code, legal_city, legal_street, legal_house,
bank_postal_code, bank_city, bank_street, bank_house, bank_account_number)
VALUES
('SUP-001', 'ООО МеталлСнаб', '7708123456', '125047', 'Москва', 'Лесная', '5',
'101000', 'Москва', 'Мясницкая', '12', '40702810900000000001'),
('SUP-002', 'АО ПромКомплект', '7816123001', '190000', 'Санкт-Петербург', 'Садовая', '21',
'101000', 'Москва', 'Мясницкая', '12', '40702810900000000002'),
('SUP-003', 'ИП Волокова Н.А.', '502900771802', '141008', 'Мытищи', 'Мира', '18',
'141002', 'Мытищи', 'Новомытищинский', '31', '40802810400000000003');

INSERT INTO materials
(material_code, material_class_code, material_group_code, material_name, material_account)

```

```

VALUES
    ('MAT-001', '10', '101', 'Лист стальной 2 мм', '10.01'),
    ('MAT-002', '20', '204', 'Краска акриловая белая', '10.06'),
    ('MAT-003', '30', '302', 'Кабель силовой ВВГнг', '10.05');

INSERT INTO measurement_units (material_code, unit_code, unit_name)
VALUES
    ('MAT-001', 'KG', 'килограммы'),
    ('MAT-001', 'PCS', 'штуки'),
    ('MAT-002', 'L', 'литры'),
    ('MAT-003', 'M', 'метры');

INSERT INTO document_types (document_type_code, document_type_name)
VALUES
    ('INV', 'Счет-фактура'),
    ('WAY', 'Товарная накладная'),
    ('ACT', 'Акт приемки');

INSERT INTO storage_units
    (order_number, order_date, supplier_code, balance_account, document_type_code, document_number,
     material_code, material_account, unit_code, quantity, unit_price)
VALUES
    (1001, '2026-04-08', 'SUP-001', '15', 'WAY', 'TH-450', 'MAT-001', '10.01', 'KG', 1200, 88.50),
    (1002, '2026-04-12', 'SUP-002', '15', 'INV', 'СФ-981', 'MAT-001', '10.01', 'PCS', 350, 145.00),
    (1003, '2026-04-19', 'SUP-003', '15', 'WAY', 'TH-118', 'MAT-002', '10.06', 'L', 90, 520.00),
    (1004, '2026-04-28', 'SUP-002', '15', 'ACT', 'АП-77', 'MAT-003', '10.05', 'M', 800, 74.30);

CREATE OR REPLACE FUNCTION count_suppliers_for_material(p_material_code varchar)
RETURNS integer
LANGUAGE sql
STABLE
AS $$
    SELECT COUNT(DISTINCT supplier_code)::integer
    FROM storage_units
    WHERE material_code = p_material_code;
$$;

CREATE OR REPLACE FUNCTION count_suppliers_by_bank_address(
    p_bank_postal_code varchar,
    p_bank_city varchar,
    p_bank_street varchar,
    p_bank_house varchar
)
RETURNS integer
LANGUAGE sql
STABLE
AS $$
    SELECT COUNT(*)::integer
    FROM suppliers
    WHERE bank_postal_code = p_bank_postal_code
        AND bank_city = p_bank_city
        AND bank_street = p_bank_street
        AND bank_house = p_bank_house;
$$;

CREATE OR REPLACE FUNCTION get_suppliers_for_material(p_material_code varchar)
RETURNS TABLE
(
    material_code varchar,
    material_name varchar,
    supplier_code varchar,
    supplier_name varchar,
    inn varchar,
    legal_postal_code varchar,
    legal_city varchar,
    legal_street varchar,
    legal_house varchar,
    bank_postal_code varchar,
    bank_city varchar,
    bank_street varchar,
    bank_house varchar,
    bank_account_number varchar,
    total_quantity numeric,

```

```

        total_amount numeric
    )
LANGUAGE sql
STABLE
AS $$
    SELECT
        m.material_code,
        m.material_name,
        s.supplier_code,
        s.supplier_name,
        s.inn,
        s.legal_postal_code,
        s.legal_city,
        s.legal_street,
        s.legal_house,
        s.bank_postal_code,
        s.bank_city,
        s.bank_street,
        s.bank_house,
        s.bank_account_number,
        SUM(u.quantity) AS total_quantity,
        SUM(u.total_price) AS total_amount
    FROM storage_units AS u
    JOIN materials AS m ON m.material_code = u.material_code
    JOIN suppliers AS s ON s.supplier_code = u.supplier_code
    WHERE u.material_code = p_material_code
    GROUP BY
        m.material_code, m.material_name, s.supplier_code, s.supplier_name, s.inn,
        s.legal_postal_code, s.legal_city, s.legal_street, s.legal_house,
        s.bank_postal_code, s.bank_city, s.bank_street, s.bank_house, s.bank_account_number
    ORDER BY s.supplier_name;
$$;

CREATE OR REPLACE PROCEDURE add_storage_unit(
    p_order_number integer,
    p_order_date date,
    p_supplier_code varchar,
    p_balance_account varchar,
    p_document_type_code varchar,
    p_document_number varchar,
    p_material_code varchar,
    p_material_account varchar,
    p_unit_code varchar,
    p_quantity numeric,
    p_unit_price numeric
)
LANGUAGE plpgsql
AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM materials
        WHERE material_code = p_material_code
        AND material_account = p_material_account
    ) THEN
        RAISE EXCEPTION 'Счет материала не соответствует справочнику материалов.';
    END IF;

    INSERT INTO storage_units
        (order_number, order_date, supplier_code, balance_account, document_type_code, document_number,
        material_code, material_account, unit_code, quantity, unit_price)
    VALUES
        (p_order_number, p_order_date, p_supplier_code, p_balance_account, p_document_type_code,
        p_document_number, p_material_code, p_material_account, p_unit_code, p_quantity, p_unit_price);
END;
$$;

CREATE OR REPLACE FUNCTION sync_storage_material_account()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.material_account IS DISTINCT FROM OLD.material_account THEN
        UPDATE storage_units
        SET material_account = NEW.material_account
    END IF;
END;
$$;

```

```

        WHERE material_code = NEW.material_code;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_materials_cascade_account_change
AFTER UPDATE OF material_account ON materials
FOR EACH ROW
EXECUTE FUNCTION sync_storage_material_account();

CREATE OR REPLACE FUNCTION delete_storage_units_for_measurement_unit()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    DELETE FROM storage_units
    WHERE material_code = OLD.material_code
    AND unit_code = OLD.unit_code;

    RETURN OLD;
END;
$$;

CREATE TRIGGER trg_measurement_units_cascade_delete
BEFORE DELETE ON measurement_units
FOR EACH ROW
EXECUTE FUNCTION delete_storage_units_for_measurement_unit();

CREATE OR REPLACE FUNCTION block_supplier_delete_with_receipts()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF EXISTS (
        SELECT 1
        FROM storage_units
        WHERE supplier_code = OLD.supplier_code
    ) THEN
        RAISE EXCEPTION 'Нельзя удалить поставщика, у которого есть складские приходы.';
    END IF;

    RETURN OLD;
END;
$$;

CREATE TRIGGER trg_suppliers_block_delete_with_receipts
BEFORE DELETE ON suppliers
FOR EACH ROW
EXECUTE FUNCTION block_supplier_delete_with_receipts();

SELECT count_suppliers_for_material('MAT-001') AS suppliers_for_mat_001;
SELECT count_suppliers_by_bank_address('101000', 'Москва', 'Мясницкая', '12') AS suppliers_for_bank;
SELECT * FROM get_suppliers_for_material('MAT-001');
```

Приложение В - Исходный код клиентской части

Листинг В.1 - App.tsx

```

import type { FormEvent, ReactNode } from 'react'
import { useEffect, useMemo, useState } from 'react'
import './App.css'

const API_URL = import.meta.env.VITE_API_URL ?? 'http://localhost:5064/api/warehouse'
const AUTH_URL = API_URL.replace(/\/api\/warehouse$/, '/api/auth')
```

```

const TOKEN_KEY = 'warehouse-auth-token'
const INVITATION_CODES = ['INVITE-2026', 'WAREHOUSE-ACCESS']

type Address = {
  postalCode: string
  city: string
  street: string
  house: string
}

type Material = {
  code: string
  classCode: string
  groupCode: string
  name: string
  materialAccount: string
}

type Supplier = {
  code: string
  name: string
  inn: string
  legalAddress: Address
  bankAddress: Address
  bankAccountNumber: string
}

type DocumentType = {
  code: string
  name: string
}

type MeasurementUnit = {
  materialCode: string
  unitCode: string
  unitName: string
}

type StorageUnit = {
  orderNumber: number
  orderDate: string
  supplierName: string
  materialName: string
  unitName: string
  quantity: number
  unitPrice: number
  totalPrice: number
  documentNumber: string
}

type SupplierForMaterial = {
  supplierCode: string
  supplierName: string
  inn: string
  legalAddress: Address
  bankAddress: Address
  bankAccountNumber: string
  totalQuantity: number
  totalAmount: number
}

type StorageUnitForm = {
  orderNumber: string
  orderDate: string
  supplierCode: string
  balanceAccount: string
  documentTypeCode: string
  documentNumber: string
  materialCode: string
  materialAccount: string
  unitCode: string
  quantity: string
}

```

```

    unitPrice: string
  }

type AuthUser = {
  email: string
  name: string
}

type AuthResponse = {
  token: string
  user: AuthUser
}

type AuthForm = {
  email: string
  name: string
  password: string
  invitationCode: string
}

const emptyAddress: Address = {
  postalCode: '',
  city: '',
  street: '',
  house: '',
}

function App() {
  const [token, setToken] = useState(() => localStorage.getItem(TOKEN_KEY) ?? '')
  const [, setCurrentUser] = useState<AuthUser | null>(null)
  const [authMode, setAuthMode] = useState<'login' | 'register'>('login')
  const [authForm, setAuthForm] = useState<AuthForm>({
    email: '',
    name: '',
    password: '',
    invitationCode: '',
  })
  const [materials, setMaterials] = useState<Material[]>([])
  const [suppliers, setSuppliers] = useState<Supplier[]>([])
  const [documents, setDocuments] = useState<DocumentType[]>([])
  const [measurementUnits, setMeasurementUnits] = useState<MeasurementUnit[]>([])
  const [storageUnits, setStorageUnits] = useState<StorageUnit[]>([])
  const [selectedMaterial, setSelectedMaterial] = useState('MAT-001')
  const [supplierReport, setSupplierReport] = useState<SupplierForMaterial[]>([])
  const [supplierCount, setSupplierCount] = useState(0)
  const [bankAddress, setBankAddress] = useState<Address>({
    postalCode: '101000',
    city: 'Москва',
    street: 'Мясницкая',
    house: '12',
  })
  const [bankCount, setBankCount] = useState<number | null>(null)
  const [isDarkTheme, setIsDarkTheme] = useState(false)
  const [isMaterialsListOpen, setIsMaterialsListOpen] = useState(false)
  const [materialSearch, setMaterialSearch] = useState('')
  const [isStorageListOpen, setIsStorageListOpen] = useState(false)
  const [storageSearch, setStorageSearch] = useState('')
  const [isSupplierReportOpen, setIsSupplierReportOpen] = useState(false)
  const [message, setMessage] = useState('')
  const [form, setForm] = useState<StorageUnitForm>({
    orderNumber: '1005',
    orderDate: new Date().toISOString().slice(0, 10),
    supplierCode: 'SUP-001',
    balanceAccount: '15',
    documentTypeCode: 'WAY',
    documentNumber: 'TH-501',
    materialCode: 'MAT-001',
    materialAccount: '10.01',
    unitCode: 'KG',
    quantity: '100',
    unitPrice: '90',
  })
}

```

```

const unitsForMaterial = useMemo(
  () => measurementUnits.filter((unit) => unit.materialCode === form.materialCode),
  [form.materialCode, measurementUnits],
)

const latestMaterials = useMemo(() => materials.slice(-4).reverse(), [materials])

const filteredMaterials = useMemo(() => {
  const search = materialSearch.trim().toLowerCase()

  if (!search) {
    return materials
  }

  return materials.filter((material) =>
    [material.code, material.classCode, material.groupCode, material.name,
    material.materialAccount].some((value) =>
      value.toLowerCase().includes(search),
    ),
  ),
  [materialSearch, materials])

const latestStorageUnits = useMemo(() => storageUnits.slice(-4).reverse(), [storageUnits])

const filteredStorageUnits = useMemo(() => {
  const search = storageSearch.trim().toLowerCase()

  if (!search) {
    return storageUnits
  }

  return storageUnits.filter((unit) =>
    [
      String(unit.orderNumber),
      formatDate(unit.orderDate),
      unit.supplierName,
      unit.materialName,
      unit.unitName,
      String(unit.quantity),
      String(unit.unitPrice),
      String(unit.totalPrice),
      unit.documentNumber,
    ].some((value) => value.toLowerCase().includes(search)),
  ),
  [storageSearch, storageUnits])

const visibleSupplierReport = useMemo(
  () => (isSupplierReportOpen ? supplierReport : supplierReport.slice(0, 4)),
  [isSupplierReportOpen, supplierReport],
)

useEffect(() => {
  if (!token) {
    setCurrentUser(null)
    return
  }

  localStorage.setItem(TOKEN_KEY, token)
  void loadCurrentUser(token)
  void loadInitialData(token)
}, [token])

useEffect(() => {
  document.documentElement.dataset.theme = isDarkTheme ? 'dark' : 'light'
}, [isDarkTheme])

useEffect(() => {
  const material = materials.find((item) => item.code === form.materialCode)

```

```

const firstUnit = measurementUnits.find((unit) => unit.materialCode === form.materialCode)

setForm((current) => ({
  ...current,
  materialAccount: material?.materialAccount ?? current.materialAccount,
  unitCode: firstUnit?.unitCode ?? current.unitCode,
}))
}, [form.materialCode, materials, measurementUnits])

useEffect(() => {
  if (token && selectedMaterial) {
    void loadMaterialReport(selectedMaterial, token)
  }
}, [selectedMaterial, token])

async function loadCurrentUser(authToken: string) {
  try {
    setCurrentUser(await authApi<AuthUser>('/me', undefined, authToken))
  } catch {
    handleLogout()
  }
}

async function loadInitialData(authToken = token) {
  try {
    const [materialsData, suppliersData, documentsData, unitsData, storageData] = await Promise.all([
      api<Material[]>('/materials', undefined, authToken),
      api<Supplier[]>('/suppliers', undefined, authToken),
      api<DocumentType[]>('/documents', undefined, authToken),
      api<MeasurementUnit[]>('/measurement-units', undefined, authToken),
      api<StorageUnit[]>('/storage-units', undefined, authToken),
    ])

    setMaterials(materialsData)
    setSuppliers(suppliersData)
    setDocuments(documentsData)
    setMeasurementUnits(unitsData)
    setStorageUnits(storageData)
  } catch (error) {
    setMessage(error instanceof Error ? error.message : 'Не удалось загрузить данные')
  }
}

async function loadMaterialReport(materialCode: string, authToken = token) {
  if (!authToken) {
    return
  }

  try {
    const [suppliersData, countData] = await Promise.all([
      api<SupplierForMaterial[]>(`/materials/${materialCode}/suppliers`, undefined, authToken),
      api<{ count: number }>(`/materials/${materialCode}/supplier-count`, undefined, authToken),
    ])
    setSupplierReport(suppliersData)
    setSupplierCount(countData.count)
  } catch {
    setSupplierReport([])
    setSupplierCount(0)
  }
}

async function handleStorageSubmit(event: FormEvent<HTMLFormElement>) {
  event.preventDefault()

  try {
    await api(
      '/storage-units',
      {
        method: 'POST',
        body: JSON.stringify({
          ...form,

```

```

        orderNumber: Number(form.orderNumber),
        quantity: Number(form.quantity),
        unitPrice: Number(form.unitPrice),
    })),
    },
    token,
)
setMessage('Единица хранения добавлена')
await loadInitialData()
await loadMaterialReport(selectedMaterial)
} catch (error) {
    setMessage(error instanceof Error ? error.message : 'Ошибка добавления')
}
}

async function handleBankSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault()

    try {
        const result = await api<{ count: number }>(
            '/suppliers/bank-address-count',
            {
                method: 'POST',
                body: JSON.stringify(bankAddress),
            },
            token,
        )
        setBankCount(result.count)
    } catch (error) {
        setMessage(error instanceof Error ? error.message : 'Ошибка расчета')
    }
}

async function handleAuthSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault()

    try {
        const result = await authApi<AuthResponse>(authMode === 'login' ? '/login' : '/register', {
            method: 'POST',
            body: JSON.stringify(
                authMode === 'login'
                ? { email: authForm.email, password: authForm.password }
                : authForm,
            ),
        })

        setCurrentUser(result.user)
        setToken(result.token)
        setMessage('')
    } catch (error) {
        setMessage(error instanceof Error ? error.message : 'Не удалось выполнить вход')
    }
}

function handleLogout() {
    localStorage.removeItem(TOKEN_KEY)
    setToken('')
    setCurrentUser(null)
    setMaterials([])
    setSuppliers([])
    setDocuments([])
    setMeasurementUnits([])
    setStorageUnits([])
    setSupplierReport([])
    setSupplierCount(0)
    setMessage('Войдите, чтобы работать со складом')
}

const selectedMaterialName =
    materials.find((material) => material.code === selectedMaterial)?.name ?? selectedMaterial

if (!token) {

```

```

return (
  <main className="app-shell auth-shell">
    <section className="panel auth-panel">
      <div className="panel-heading">
        <div>
          <h1>Складской учет</h1>
          <p>{authMode === 'login' ? 'Вход для сотрудников' : 'Регистрация по приглашению'}</p>
        </div>
        <button
          type="button"
          onClick={() => setAuthMode((current) => (current === 'login' ? 'register' : 'login'))}
        >
          {authMode === 'login' ? 'Регистрация' : 'Войти'}
        </button>
      </div>

      <form className="auth-form" onSubmit={handleAuthSubmit}>
        <Field label="Почта">
          <input
            type="email"
            value={authForm.email}
            onChange={(event) => setAuthFormValue('email', event.target.value)}
            required
          />
        </Field>
        {authMode === 'register' && (
          <Field label="Имя">
            <input
              value={authForm.name}
              onChange={(event) => setAuthFormValue('name', event.target.value)}
              required
            />
          </Field>
        )}
        <Field label="Пароль">
          <input
            type="password"
            value={authForm.password}
            onChange={(event) => setAuthFormValue('password', event.target.value)}
            required
          />
        </Field>
        {authMode === 'register' && (
          <Field label="Код приглашения">
            <input
              value={authForm.invitationCode}
              onChange={(event) => setAuthFormValue('invitationCode', event.target.value)}
              required
            />
          </Field>
        )}
        <button type="submit">{authMode === 'login' ? 'Войти' : 'Создать аккаунт'}</button>
        {message && <span className="status">{message}</span>}
      </form>
    </section>

    {authMode === 'register' && (
      <aside className="panel invitation-panel">
        <div className="panel-heading">
          <div>
            <h2>Коды приглашения</h2>
            <p>Нажмите на код, чтобы подставить его в форму регистрации</p>
          </div>
        </div>
        <div className="invitation-codes">
          {INVITATION_CODES.map((code) => (
            <button
              key={code}
              type="button"
              onClick={() => setAuthFormValue('invitationCode', code)}
            >
              {code}
            </button>
          ))}
        </div>
      </aside>
    )}
  </main>

```

```

    })
  </main>
)
}

return (
  <main className="app-shell">
    <header className="topbar">
      <div>
        <h1>Складской учет материалов</h1>
      </div>
      <div className="topbar-actions">
        <button
          className="theme-toggle"
          type="button"
          aria-pressed={isDarkTheme}
          onClick={() => setIsDarkTheme((current) => !current)}
        >
          {isDarkTheme ? 'Светлая тема' : 'Темная тема'}
        </button>
        <button type="button" onClick={handleLogout}>Выйти</button>
        {message && <span className="status">{message}</span>}
      </div>
    </header>

    <section className="metrics" aria-label="Показатели склада">
      <Metric label="Материалов" value={materials.length} />
      <Metric label="Поставщиков" value={suppliers.length} />
      <Metric label="Единиц хранения" value={storageUnits.length} />
      <Metric
        label="Остаток по приходу"
        value={formatMoney(storageUnits.reduce((sum, unit) => sum + unit.totalPrice, 0))}
      />
    </section>

    <section className="panel materials-panel">
      <div className="panel-heading">
        <div>
          <h2>Последние материалы</h2>
          <p>Показаны 4 последних материала из справочника</p>
        </div>
        <button type="button" onClick={() => setIsMaterialsListOpen((current) => !current)}>
          {isMaterialsListOpen ? 'Скрыть все материалы' : 'Ко всем материалам'}
        </button>
      </div>

      <div className="materials-preview">
        {latestMaterials.map((material) => (
          <article className="material-card" key={material.code}>
            <strong>{material.name}</strong>
            <span>Код: {material.code}</span>
            <span>Класс: {material.classCode}</span>
            <span>Группа: {material.groupCode}</span>
            <b>Счет: {material.materialAccount}</b>
          </article>
        ))}
      </div>

      {isMaterialsListOpen && (
        <div className="materials-all">
          <Field label="Поиск по материалам">
            <input
              placeholder="Название, код, класс, группа или счет"
              value={materialSearch}
              onChange={(event) => setMaterialSearch(event.target.value)}
            />
          </Field>
          <div className="table-wrap">
            <table className="materials-table">
              <thead>
                <tr>
                  <th>Код</th>
                  <th>Наименование</th>
                  <th>Класс</th>
                </tr>
              </thead>
            </table>
          </div>
        </div>
      )}
    </section>
  </main>
)

```

```

        <th>Группа</th>
        <th>Счет</th>
      </tr>
    </thead>
    <tbody>
      {filteredMaterials.map((material) => (
        <tr key={material.code}>
          <td>{material.code}</td>
          <td>{material.name}</td>
          <td>{material.classCode}</td>
          <td>{material.groupCode}</td>
          <td>{material.materialAccount}</td>
        </tr>
      ))}
    </tbody>
  </table>
</div>
</div>
))
</section>

<section className="workspace">
  <form className="panel form-panel" onSubmit={handleStorageSubmit}>
    <div className="panel-heading">
      <h2>Добавление единицы хранения</h2>
      <button type="submit">Добавить</button>
    </div>

    <div className="form-grid">
      <Field label="Номер ордера">
        <input value={form.orderNumber} onChange={(event) => setFormValue('orderNumber',
event.target.value)} />
      </Field>
      <Field label="Дата">
        <input type="date" value={form.orderDate} onChange={(event) => setFormValue('orderDate',
event.target.value)} />
      </Field>
      <Field label="Поставщик">
        <select value={form.supplierCode} onChange={(event) => setFormValue('supplierCode',
event.target.value)}>
          {suppliers.map((supplier) => (
            <option key={supplier.code} value={supplier.code}>
              {supplier.name}
            </option>
          ))}
        </select>
      </Field>
      <Field label="Балансовый счет">
        <input value={form.balanceAccount} onChange={(event) => setFormValue('balanceAccount',
event.target.value)} />
      </Field>
      <Field label="Документ">
        <select
          value={form.documentTypeCode}
          onChange={(event) => setFormValue('documentTypeCode', event.target.value)}
        >
          {documents.map((document) => (
            <option key={document.code} value={document.code}>
              {document.name}
            </option>
          ))}
        </select>
      </Field>
      <Field label="Номер документа">
        <input value={form.documentNumber} onChange={(event) => setFormValue('documentNumber',
event.target.value)} />
      </Field>
      <Field label="Материал">
        <select value={form.materialCode} onChange={(event) => setFormValue('materialCode',
event.target.value)}>
          {materials.map((material) => (
            <option key={material.code} value={material.code}>
              {material.name}
            </option>
          ))}
        </select>
      </Field>
    </div>
  </form>
</section>

```

```

        </Field>
        <Field label="Счет материала">
          <input value={form.materialAccount} onChange={(event) => setFormValue('materialAccount',
event.target.value)} />
        </Field>
        <Field label="Единица измерения">
          <select value={form.unitCode} onChange={(event) => setFormValue('unitCode',
event.target.value)}>
            {unitsForMaterial.map((unit) => (
              <option key={unit.unitCode} value={unit.unitCode}>
                {unit.unitName}
              </option>
            ))}
          </select>
        </Field>
        <Field label="Количество">
          <input value={form.quantity} onChange={(event) => setFormValue('quantity', event.target.value)}
        />
        </Field>
        <Field label="Цена">
          <input value={form.unitPrice} onChange={(event) => setFormValue('unitPrice',
event.target.value)} />
        </Field>
      </div>
    </form>

    <section className="panel">
      <div className="panel-heading">
        <div>
          <h2>Приход на склад</h2>
          <p>Показаны 4 последних прихода</p>
        </div>
        <button type="button" onClick={() => setIsStorageListOpen((current) => !current)}>
          {isStorageListOpen ? 'Скрыть весь приход' : 'Ко всему приходу'}
        </button>
      </div>
      <div className="table-wrap">
        <table>
          <thead>
            <tr>
              <th>Ордер</th>
              <th>Дата</th>
              <th>Поставщик</th>
              <th>Материал</th>
              <th>Кол-во</th>
              <th>Сумма</th>
            </tr>
          </thead>
          <tbody>
            {latestStorageUnits.map((unit) => (
              <tr key={unit.orderNumber}>
                <td>{unit.orderNumber}</td>
                <td>{formatDate(unit.orderDate)}</td>
                <td>{unit.supplierName}</td>
                <td>{unit.materialName}</td>
                <td>
                  {unit.quantity} {unit.unitName}
                </td>
                <td>{formatMoney(unit.totalPrice)}</td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>

      {isStorageListOpen && (
        <div className="storage-all">
          <Field label="Поиск по приходу">
            <input
              placeholder="Ордер, дата, поставщик, материал, документ или сумма"
              value={storageSearch}
              onChange={(event) => setStorageSearch(event.target.value)}
            />
          </Field>
        <div className="table-wrap">
          <table>

```

```

        <thead>
        <tr>
            <th>Ордер</th>
            <th>Дата</th>
            <th>Поставщик</th>
            <th>Материал</th>
            <th>Документ</th>
            <th>Кол-во</th>
            <th>Цена</th>
            <th>Сумма</th>
        </tr>
        </thead>
        <tbody>
            {filteredStorageUnits.map((unit) => (
                <tr key={unit.orderNumber}>
                    <td>{unit.orderNumber}</td>
                    <td>{formatDate(unit.orderDate)}</td>
                    <td>{unit.supplierName}</td>
                    <td>{unit.materialName}</td>
                    <td>{unit.documentNumber}</td>
                    <td>
                        {unit.quantity} {unit.unitName}
                    </td>
                    <td>{formatMoney(unit.unitPrice)}</td>
                    <td>{formatMoney(unit.totalPrice)}</td>
                </tr>
            ))}
        </tbody>
    </table>
</div>
</div>
)}
</section>
</section>

<section className="reports">
    <section className="panel">
        <div className="panel-heading">
            <div>
                <h2>Поставщики материала</h2>
                <p>{supplierCount} поставщика для: {selectedMaterialName}</p>
            </div>
            <div className="report-actions">
                <Field label="Материал для отчета">
                    <select
                        value={selectedMaterial}
                        onChange={(event) =>
setSelectedMaterial(event.target.value)}
                    >
                        {materials.map((material) => (
                            <option key={material.code} value={material.code}>
                                {material.name}
                            </option>
                        ))}
                    </select>
                </Field>
                <button type="button" onClick={() => setIsSupplierReportOpen((current) => !current)}>
                    {isSupplierReportOpen ? 'Скрыть всех поставщиков' : 'Ко всем поставщикам'}
                </button>
            </div>
        </div>
        <div className="supplier-list">
            {visibleSupplierReport.map((supplier) => (
                <article className="supplier-card" key={supplier.supplierCode}>
                    <strong>{supplier.supplierName}</strong>
                    <span>ИНН {supplier.inn}</span>
                    <span>{formatAddress(supplier.legalAddress)}</span>
                    <span>Банк: {formatAddress(supplier.bankAddress)}</span>
                    <span>Счет: {supplier.bankAccountNumber}</span>
                    <b>{formatMoney(supplier.totalAmount)}</b>
                </article>
            ))}
        </div>
    </section>

    <form className="panel" onSubmit={handleBankSubmit}>
        <div className="panel-heading">
            <div>
                <h2>Поставщики по адресу банка</h2>

```

```

        <p>{bankCount === null ? 'Введите адрес банка' : `Найдено: ${bankCount}`}</p>
      </div>
      <button type="submit">Посчитать</button>
    </div>
    <div className="address-grid">
      <Field label="Индекс банка">
        <input
          placeholder="101000"
          value={bankAddress.postalCode}
          onChange={(event) => setBankAddressValue('postalCode', event.target.value)}
        />
      </Field>
      <Field label="Город банка">
        <input
          placeholder="Москва"
          value={bankAddress.city}
          onChange={(event) => setBankAddressValue('city', event.target.value)}
        />
      </Field>
      <Field label="Улица банка">
        <input
          placeholder="Мясницкая"
          value={bankAddress.street}
          onChange={(event) => setBankAddressValue('street', event.target.value)}
        />
      </Field>
      <Field label="Дом банка">
        <input
          placeholder="12"
          value={bankAddress.house}
          onChange={(event) => setBankAddressValue('house', event.target.value)}
        />
      </Field>
    </div>
  </form>
</section>
</main>
)

function setFormValue(field: keyof StorageUnitForm, value: string) {
  setForm((current) => ({ ...current, [field]: value }))
}

function setBankAddressValue(field: keyof Address, value: string) {
  setBankAddress((current) => ({ ...current, [field]: value }))
}

function setAuthFormValue(field: keyof AuthForm, value: string) {
  setAuthForm((current) => ({ ...current, [field]: value }))
}

function Field({ label, children }: { label: string; children: ReactNode }) {
  return (
    <label className="field">
      <span>{label}</span>
      {children}
    </label>
  )
}

function Metric({ label, value }: { label: string; value: string | number }) {
  return (
    <article className="metric">
      <span>{label}</span>
      <strong>{value}</strong>
    </article>
  )
}

async function api<T>(path: string, init?: RequestInit, token?: string): Promise<T> {
  const response = await fetch(`${API_URL}${path}`, {
    headers: {

```

```

        'Content-Type': 'application/json',
        ...(token ? { Authorization: `Bearer ${token}` } : {}),
        ...init?.headers,
    },
    ...init,
  })

  if (!response.ok) {
    const error = await response.json().catch(() => null)
    throw new Error(error?.message ?? 'Сервер вернул ошибку')
  }

  return response.json()
}

async function authApi<T>(path: string, init?: RequestInit, token?: string): Promise<T> {
  const response = await fetch(`${AUTH_URL}${path}`, {
    headers: {
      'Content-Type': 'application/json',
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
      ...init?.headers,
    },
    ...init,
  })

  if (!response.ok) {
    const error = await response.json().catch(() => null)
    throw new Error(error?.message ?? 'Сервер вернул ошибку')
  }

  return response.json()
}

function formatAddress(address: Address = emptyAddress) {
  return `${address.postalCode}, ${address.city}, ${address.street}, ${address.house}`
}

function formatDate(value: string) {
  return new Intl.DateTimeFormat('ru-RU').format(new Date(value))
}

function formatMoney(value: number) {
  return new Intl.NumberFormat('ru-RU', {
    style: 'currency',
    currency: 'RUB',
    maximumFractionDigits: 0,
  }).format(value)
}

export default App

```

Листинг В.2 - App.css

```

.app-shell {
  width: min(1440px, 100%);
  margin: 0 auto;
  padding: 28px;
}

.topbar {
  display: flex;
  align-items: flex-end;
  justify-content: space-between;
  gap: 24px;
  padding-bottom: 24px;
  border-bottom: 1px solid var(--line);
}

.topbar-actions {
  display: flex;

```

```

    align-items: center;
    gap: 12px;
}

.auth-shell {
  display: grid;
  gap: 16px;
  min-height: 100vh;
  place-items: center;
}

.auth-panel {
  width: min(520px, 100%);
}

.invitation-panel {
  width: min(520px, 100%);
}

.auth-form {
  display: grid;
  gap: 14px;
  padding: 18px;
}

.invitation-codes {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
  padding: 12px;
}

.invitation-codes button {
  padding: 8px 10px;
  font-family: inherit;
  font-size: 13px;
}

.eyebrow {
  margin: 0 0 6px;
  color: var(--muted);
  font-size: 13px;
  font-weight: 700;
  letter-spacing: 0;
  text-transform: uppercase;
}

.status {
  min-width: 220px;
  padding: 9px 12px;
  border: 1px solid var(--line);
  border-radius: 8px;
  color: var(--ink);
  background: var(--panel-muted);
  text-align: center;
}

.metrics {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 12px;
  margin: 24px 0;
}

.metric {
  padding: 18px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--surface);
}

```

```

.metric span {
  display: block;
  color: var(--muted);
  font-size: 14px;
}

.metric strong {
  display: block;
  margin-top: 8px;
  color: var(--ink);
  font-size: 28px;
  line-height: 1.1;
}

.materials-panel {
  margin-bottom: 18px;
}

.materials-preview {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 12px;
  padding: 18px;
}

.material-card {
  display: grid;
  gap: 5px;
  min-height: 136px;
  padding: 14px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--field-bg);
}

.material-card strong,
.material-card b {
  color: var(--ink);
}

.material-card span {
  color: var(--muted);
  font-size: 14px;
}

.materials-all {
  display: grid;
  gap: 14px;
  padding: 0 18px 18px;
}

.materials-all .table-wrap,
.storage-all .table-wrap {
  max-height: 420px;
  border: 1px solid var(--line);
  border-radius: 8px;
}

.materials-table {
  min-width: 680px;
}

.storage-all {
  display: grid;
  gap: 14px;
  padding: 18px;
  border-top: 1px solid var(--line);
}

```

```

.workspace,
.reports {
  display: grid;
  grid-template-columns: minmax(360px, 0.85fr) minmax(520px, 1.15fr);
  gap: 18px;
  align-items: start;
}

.reports {
  grid-template-columns: 1.2fr 0.8fr;
  margin-top: 18px;
}

.panel {
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--surface);
  overflow: hidden;
}

.panel-heading {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  padding: 16px 18px;
  border-bottom: 1px solid var(--line);
  background: var(--panel-muted);
}

.panel-heading p {
  margin-top: 4px;
  color: var(--muted);
}

.form-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 12px;
  padding: 18px;
}

.field {
  display: grid;
  gap: 6px;
  min-width: 0;
}

.field span {
  color: var(--muted);
  font-size: 13px;
  font-weight: 700;
}

input,
select,
button {
  min-height: 40px;
  border: 1px solid var(--line-strong);
  border-radius: 7px;
  font: inherit;
}

input,
select {
  width: 100%;
  box-sizing: border-box;
  padding: 8px 10px;
  color: var(--ink);
}

```

```

    background: var(--field-bg);
}

button {
  padding: 8px 14px;
  color: var(--accent-ink);
  background: var(--accent);
  border-color: var(--accent);
  font-weight: 700;
  cursor: pointer;
}

button:hover {
  background: var(--accent-hover);
  border-color: var(--accent-hover);
}

.theme-toggle {
  white-space: nowrap;
}

.report-actions {
  display: flex;
  align-items: end;
  gap: 10px;
}

.report-actions .field {
  min-width: 240px;
}

.table-wrap {
  overflow-x: auto;
}

table {
  width: 100%;
  border-collapse: collapse;
  min-width: 760px;
}

th,
td {
  padding: 13px 14px;
  border-bottom: 1px solid var(--line);
  text-align: left;
  vertical-align: top;
}

th {
  color: var(--muted);
  font-size: 13px;
  font-weight: 700;
  background: var(--panel-muted);
}

td {
  color: var(--ink);
}

.supplier-list {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 12px;
  padding: 18px;
}

.supplier-card {
  display: grid;

```

```

    gap: 5px;
    padding: 14px;
    border: 1px solid var(--line);
    border-radius: 8px;
    background: var(--field-bg);
}

.supplier-card strong,
.supplier-card b {
    color: var(--ink);
}

.supplier-card span {
    color: var(--muted);
    font-size: 14px;
}

.address-grid {
    display: grid;
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: 12px;
    padding: 18px;
}

@media (max-width: 1100px) {
    .workspace,
    .reports,
    .metrics,
    .materials-preview {
        grid-template-columns: 1fr;
    }
}

@media (max-width: 700px) {
    .app-shell {
        padding: 16px;
    }

    .topbar,
    .panel-heading {
        align-items: stretch;
        flex-direction: column;
    }

    .topbar-actions {
        align-items: stretch;
        flex-direction: column;
    }

    .report-actions {
        align-items: stretch;
        flex-direction: column;
    }

    .report-actions .field {
        min-width: 0;
    }

    .form-grid,
    .supplier-list,
    .address-grid {
        grid-template-columns: 1fr;
    }
}

```

Листинг В.3 - package.json

```

{
  "name": "warehouse-client",
  "private": true,
  "version": "0.0.0",

```

```

    "type": "module",
    "scripts": {
      "dev": "vite",
      "build": "tsc -b && vite build",
      "lint": "eslint .",
      "preview": "vite preview"
    },
    "dependencies": {
      "react": "^19.2.5",
      "react-dom": "^19.2.5"
    },
    "devDependencies": {
      "@eslint/js": "^10.0.1",
      "@types/node": "^24.12.2",
      "@types/react": "^19.2.14",
      "@types/react-dom": "^19.2.3",
      "@vitejs/plugin-react": "^6.0.1",
      "eslint": "^10.2.1",
      "eslint-plugin-react-hooks": "^7.1.1",
      "eslint-plugin-react-refresh": "^0.5.2",
      "globals": "^17.5.0",
      "typescript": "~6.0.2",
      "typescript-eslint": "^8.58.2",
      "vite": "^8.0.10"
    }
  }
}

```